5주차: 힙·구간·그래프 탐색

측정값의 우선순위와 장애 확산

측정값이 계속 들어오는 동안 상위 k개를 유지하고, 겹치는 정비 시간을 합쳐야 합니다. 센서 지도에서는 떨어진 장애 영역과 장애가 퍼지는 시간을 구합니다.

전체 정렬·부분 순위·연결 관계는 서로 다른 상태가 필요합니다. 힙의 경계, 정렬된 구간의 끝, 탐색의 방문 기록을 추적합니다.

5주차 학습 목표

9·10차시: 힙 경계·구간 합집합·방문 집합·BFS 레벨

- 입력 계약과 경계 조건 확인
- 상태 추적과 Python 구현
- 정당성·시간·보조 공간·출력 공간 설명

도입 상황, 표와 점검 문제는 강의용 추가 예시입니다.

9차시 · 정렬·힙·구간

계속 들어오는 수치의 상위 k개와 겹치는 정비 구간을 관리합니다. 모든 값을 완전 정렬해야 하는 질문과 최소 경계 하나만 필요한 질문을 구분합니다.

선수 지식과 Python 문법

sorted(items,key=lambda item:item[0])는 첫 좌표로 정렬합니다. heapq는 최소 힙입니다. heapify는 O(n), push/pop은 O(log n)입니다. tuple 우선순위가 같으면 다음 원소도 비교하므로 비교 불가 객체 앞에 고유 카운터를 둡니다.

완전 정렬과 부분 순위

- 전체 정렬: O(n log n) 시간, 모든 순위 확인
- 크기 k 최소 힙: 상위 k개만 저장
- 새 값 ≤ 루트: 기존 상위 k개 유지
- 새 값 > 루트: 루트 교체 후 힙 복구

`heapq`를 가져옵니다. heapify는 O(k), 교체는 O(log(k+1))이며 k=1도 처리합니다.

215 · 문제와 상태

LeetCode 215: Kth Largest Element in an Array · Medium

정렬했을 때 큰 순서로 k번째 값을 반환합니다. 중복도 개수에 포함합니다.

`nums=[8,3,8,2,5], k=2 → 8`

처음 k개를 heapify하고 이후 값이 root보다 크면 heapreplace합니다.

215 · 구현

```
def kth_largest(nums, k):
    heap = nums[:k]
    heapq.heapify(heap)
    for index in range(k, len(nums)):
        value = nums[index]
        if value > heap[0]:
            heapq.heapreplace(heap, value)
    return heap[0]
```

215 · 정당성과 비용

힙은 지금까지 본 값 중 큰 k개입니다. root 이하의 새 값은 상위 k개를 바꾸지 못하고, 큰 새 값은 가장 작은 보관 값을 대체해야 합니다. 마지막 root가 k번째입니다.

n=길이. O(n log(k+1)) 시간, O(k) 보조 공간. 초기 heapify O(k). 전체 음수 힙 대안은 O(n+k log n) 시간·O(n) 공간.

56 · 문제와 상태

LeetCode 56: Merge Intervals · Medium

닫힌 구간들의 합집합을 겹치지 않는 구간으로 반환합니다.

`intervals=[[5,7],[1,3],[3,6]] → [[1,7]]`

시작점으로 정렬하고 마지막 결과 구간과 겹치면 끝을 max로 늘립니다.

56 · 구현

```
def merge_intervals(intervals):
    result = []
    for start, end in sorted(intervals, key=lambda item: item[0]):
        if result and start <= result[-1][1]:
            result[-1][1] = max(result[-1][1], end)
        else:
            result.append([start, end])
    return result
```

56 · 정당성과 비용

처리한 접두부의 합집합은 result와 같습니다. 시작점이 증가하므로 새 구간은 마지막 결과와만 겹칠 수 있습니다. 분리되면 이후에도 앞 결과와 겹치지 않습니다.

n=구간 수. 정렬 O(n log n), 스캔 O(n); 정렬 보조 O(n), 출력 O(n). 입력을 복사 정렬합니다.

상태 추적

입력 값(k=2)	보관 값(중복 포함)	root
8,3	{3,8}	3
8	{8,8}	8
2	{8,8}	8
5	{8,8}	8

각 행은 지금까지 본 값 중 큰 두 개를 중복 포함해 유지합니다.

실패 입력과 이해 확인

최소 힙에서는 각 부모의 값이 자식 이하이고, root에 전체 최솟값이 놓입니다. 중복을 set으로 없애면 예시의 답이 달라집니다.

정렬된 배열에 반복 삽입하는 비용도 log n인가요?

교사용 답은 별도 진행안에 있습니다. 코드 설명은 읽기자료의 구현과 줄별로 대조합니다.

10차시 · 그래프 표현과 동시 전파

센서 격자의 연결된 영역을 세고 여러 시작점에서 번지는 장애 시간을 구합니다. 연결된 영역은 DFS나 BFS로 찾고, 간선별 전파 시간이 같을 때의 최소 도착 시간은 BFS의 레벨로 계산합니다.

선수 지식과 Python 문법

격자는 (row,col), 인접 리스트는 정점→이웃 목록, 객체 그래프는 node.neighbors로 표현합니다. 좌표 tuple은 set에 넣을 수 있습니다. 복사 지도는 [row[:] for row in grid]로 만듭니다.

200 · 문제와 상태

LeetCode 200: Number of Islands · Medium

문자 격자의 상하좌우로 연결된 육지 덩어리 수를 반환합니다.

`grid=[["1","0","1"],["1","0","0"]] → 2`

미방문 육지에서 count를 늘리고 연결된 육지를 스택으로 모두 표시합니다.

200 · 구현

```
def islands(grid):
    rows, cols = len(grid), len(grid[0])
    seen, count = set(), 0
    for row in range(rows):
        for col in range(cols):
            if grid[row][col] != "1" or (row, col) in seen:
                continue
            count += 1
            stack = [(row, col)]
            seen.add((row, col))
            while stack:
                r, c = stack.pop()
                for nr, nc in ((r-1,c),(r+1,c),(r,c-1),(r,c+1)):
                    if (0 <= nr < rows and 0 <= nc < cols
                            and grid[nr][nc] == "1" and (nr,nc) not in seen):
                        seen.add((nr,nc))
                        stack.append((nr,nc))
    return count
```

200 · 정당성과 비용

한 번 시작한 탐색은 연결 경로를 따라 그 성분의 모든 노드와 그 노드만 방문합니다. 이미 표시한 성분은 다시 시작하지 않으므로 시작 횟수가 성분 수입니다.

R,C=행·열 수. O(RC) 시간·O(RC) 보조 공간. 일반 인접 리스트에서는 O(V+E) 시간입니다.

994 · 문제와 상태

LeetCode 994: Rotting Oranges · Medium

동시에 퍼지는 변화를 적용해 모든 신선한 셀이 바뀌는 최소 분을 반환합니다. 불가능하면 -1입니다.

`grid=[[2,1,1],[0,1,0]] → 2`

모든 시작점을 같은 큐에 넣고 레벨별로 처리합니다. 새 셀은 enqueue할 때 즉시 2로 바꿉니다.

994 · 구현

```
def rot_minutes(grid):
    rows, cols = len(grid), len(grid[0])
    board = [row[:] for row in grid]
    queue, fresh, minutes = deque(), 0, 0
    for r in range(rows):
        for c in range(cols):
            if board[r][c] == 2:
                queue.append((r,c))
            fresh += board[r][c] == 1
    while queue and fresh:
        for _ in range(len(queue)):
            r, c = queue.popleft()
            for nr, nc in ((r-1,c),(r+1,c),(r,c-1),(r,c+1)):
                if 0 <= nr < rows and 0 <= nc < cols and board[nr][nc] == 1:
                    board[nr][nc] = 2
                    fresh -= 1
                    queue.append((nr,nc))
        minutes += 1
    return minutes if fresh == 0 else -1
```

994 · 정당성과 비용

큐의 레벨은 어떤 시작점에서든 그 셀까지의 최소 간선 수입니다. 모든 간선의 전파 시간이 같으므로 먼저 발견한 시간이 최소입니다. 큐가 비어도 fresh가 남으면 경로가 없습니다.

R,C=행·열 수. O(RC) 시간, 복사 board와 큐 O(RC) 보조 공간.

상태 추적

분	처리 frontier	새로 표시
0	(0,0)	(0,1)
1	(0,1)	(0,2),(1,1)
2	(0,2),(1,1)	없음

새로 표시한 셀의 시점은 t가 아니라 t+1입니다.

실패 입력과 이해 확인

값 비교는 ==입니다. is는 객체 동일성입니다. [[“1”,”0”],[“0”,”1”]]은 대각선을 연결하지 않아 두 성분입니다.

BFS가 가중 간선에서도 최소 비용을 보장하나요?

교사용 답은 별도 진행안에 있습니다. 코드 설명은 읽기자료의 구현과 줄별로 대조합니다.

동시 전파의 시간 경계

시간	그 시점에 새로 썩은 셀	남은 신선한 셀
0	(0,0), 초기 시작점	3
1	(0,1)	2
2	(0,2), (1,1)	0

격자 `[[2,1,1],[0,1,0]]`. 큐 길이를 고정한 한 번의 반복이 1분입니다. 새 셀은 큐에 넣을 때 표시합니다.

중간 점검

1. `[8,3,8,2,5]`에서 k=2와 k=5의 답은?
2. `[[1,8],[2,3]]`과 `[[1,3],[3,6]]`의 합집합은?
3. `[["1","0"],["0","1"]]`의 연결 성분 수는?
4. `[[2,0,1]]`과 `[[0,2]]`의 전파 시간은?

독립 실습과 다음 연결

학생 실습

힌트·해설

교사용 진행안

5주차 코드 내려받기

코드·상태 정의·정당성·복잡도·경계 입력을 함께 제출합니다. 8주차에는 주제를 숨긴 혼합 평가가 있습니다.

실행과 참고자료

주차별 코드의 code 폴더에서 실행합니다.

```
python demo_09.py
python demo_10.py
python verify.py
python verify.py --student
```

마지막 명령은 학생 파일을 구현한 뒤 실행합니다.

Erik Demaine·Srini Devadas, MIT 6.006: Introduction to Algorithms (2011).